

GIẢI THÍCH CODE HỆ THỐNG TURRET BẮM & BẮN LASER

Tài liệu mô tả **từng hàm / từng khối code** của hệ thống camera tự động phát hiện con chuột, xoay 2 trục (pan/tilt) bám theo, và bắn laser khi đã ngắm trúng.

Phần cứng: PC (GTX 1050 Ti) chạy thị giác máy + Arduino UNO điều khiển 2 động cơ bước closed-loop (57HSE2.2N + driver HBS57) qua tín hiệu STEP/DIR, laser.

Phần mềm: Python + OpenCV + YOLOv11-seg (Ultralytics) + pyserial.

MỤC LỤC

1. Tổng quan luồng hoạt động
2. Code giao tiếp với Arduino
3. Code đọc hình ảnh từ camera
4. Code phát hiện đối tượng (YOLO)
5. Code điều khiển pan/tilt (động cơ bước)
6. Code quét tuần tra (Scanner)
7. Code bắn laser
8. Vòng lặp chính
9. Code phía Arduino (.ino)

1. TỔNG QUAN LUỒNG HOẠT ĐỘNG

Camera → YOLO phát hiện chuột → tính sai số (dx, dy) so với tâm ngắm
→ điều khiển 2 động cơ xoay camera về phía chuột
→ khi chuột vào "vùng chết" (deadzone) và động cơ DỪNG HẸN → bắn laser
→ khi mất chuột → quét tuần tra trái/phải để tìm lại

Hệ thống có **3 trạng thái**:

Trạng thái	Khi nào	Hành vi
TRACKING	Thấy chuột	Động cơ bám theo, tốc độ tỉ lệ với sai số
WAITING	Mất chuột < 2 giây	Đứng yên chờ
SCANNING	Mất chuột ≥ 2 giây	Quét trái↔phải tìm mục tiêu

2. CODE GIAO TIẾP VỚI ARDUINO

PC nói chuyện với Arduino qua cổng Serial (COM5 trên Windows), tốc độ 9600 baud.

2.1. Khởi tạo kết nối Serial

```
import serial
import time

# Windows: "COM5" | Linux/Pi: "/dev/ttyUSB0"
PORT = "COM5"
BAUD = 9600

ser = serial.Serial(PORT, BAUD, timeout=0.1)
ser.setDTR(False)          # TẮT tín hiệu DTR -> Arduino KHÔNG bị tự reset khi mở cổng
time.sleep(2)              # chờ Arduino khởi động xong
ser.reset_input_buffer()    # xóa bộ đệm (bỏ các dòng "READY" ban đầu)

ser.write(b"M\n")          # bật chế độ Arduino báo trạng thái công tắc hành trình
ser.flush()
time.sleep(0.1)
ser.write(b"F\n")          # mặc định bật chế độ TRACK (chặn cứng khi chạm công tắc)
ser.flush()
```

Giải thích:

- `ser.setDTR(False)` : Quan trọng! Mặc định khi PC mở cổng Serial, đường DTR sẽ kéo Arduino reset lại từ đầu. Tắt nó đi để Arduino chạy liên tục.
- `"M\n"` : lệnh yêu cầu Arduino gửi về trạng thái 4 công tắc hành trình dạng `LIM:1,0,0,1` .
- `"F\n"` : lệnh chuyển Arduino sang chế độ TRACK (bám) — khi chạm công tắc thì dừng cứng để bảo vệ phần cứng.

2.2. Hàm gửi lệnh

```
def send_raw(s):
    """Gửi chuỗi thẳng tới Arduino (dùng cho lệnh tốc độ P/T)."""
    ser.write(s.encode())
    ser.flush()

def send(cmd):
    """Gửi lệnh + in log ra màn hình (dùng cho L/K/x...)."""
    print("[SEND]", cmd)
    ser.write(cmd.encode())
    ser.flush()
```

Bảng lệnh gửi cho Arduino:

Lệnh	Ý nghĩa
P<số>\n	Tốc độ động cơ PAN (ngang). VD P1200 = quay phải 1200 step/s, P-800 = trái, P0 = dừng
T<số>\n	Tốc độ động cơ TILT (dọc). T>0 = cúi xuống, T<0 = ngẩng lên
L	Bật laser
K	Tắt laser
x	Dừng hết + tắt laser
F	Chế độ TRACK (bám, chặn công tắc)
S	Chế độ SCAN (quét, không chặn)
M	Bật báo trạng thái công tắc

2.3. Hàm đọc trạng thái công tắc hành trình

```
# Biến toàn cục lưu trạng thái 4 công tắc
lim_tilt_neg = False # công tắc trên (chặn ngẩng lên)
lim_tilt_pos = False # công tắc dưới (chặn cúi xuống)
lim_pan_neg = False # công tắc trái (chặn quay trái)
lim_pan_pos = False # công tắc phải (chặn quay phải)
_serial_buf = ""

def poll_limits():
    """Đọc các dòng Arduino gửi về, cập nhật trạng thái công tắc. KHÔNG block."""
    global _serial_buf, lim_tilt_neg, lim_tilt_pos, lim_pan_neg, lim_pan_pos
    try:
        n = ser.in_waiting # số byte đang chờ đọc
    except Exception:
        return
    if n:
        _serial_buf += ser.read(n).decode(errors="ignore")
        # tách từng dòng hoàn chỉnh (kết thúc bằng '\n')
        while "\n" in _serial_buf:
            line, _serial_buf = _serial_buf.split("\n", 1)
            line = line.strip()
            if not line:
                continue
            if line.startswith("LIM:"):
                # Arduino gửi: "LIM:a0,a1,a2,a3"
                parts = line[4:].split(",")
                a0, a1, a2, a3 = (int(p) for p in parts[:4])
                lim_tilt_neg = bool(a0)
                lim_tilt_pos = bool(a1)
                lim_pan_neg = bool(a2)
                lim_pan_pos = bool(a3)
            else:
                print(f"[ARD] {line}") # các thông báo khác để debug
```

Giải thích: Hàm này đọc **không chặn** (non-blocking) — chỉ đọc khi có dữ liệu (`ser.in_waiting`), gộp vào buffer rồi tách từng dòng. Nếu dòng bắt đầu bằng LIM: thì cập nhật trạng thái 4 công tắc; còn lại in ra để debug.

3. CODE ĐỌC HÌNH ẢNH TỪ CAMERA

Dùng OpenCV (cv2) đọc webcam UVC. Chọn backend theo hệ điều hành.

3.1. Mở camera

```
import cv2
import sys

CAM_INDEX = 0          # nếu không mở được, thử 1, 2...
FRAME_W = 640
FRAME_H = 480

if sys.platform.startswith("win"):
    cam_backend = cv2.CAP_DSHOW      # Windows
else:
    cam_backend = cv2.CAP_V4L2       # Linux / Raspberry Pi

cap = cv2.VideoCapture(CAM_INDEX, cam_backend)
cap.set(cv2.CAP_PROP_FRAME_WIDTH,  FRAME_W)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_H)
cap.set(cv2.CAP_PROP_FPS, 30)
cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)  # giảm buffer -> frame không bị trễ (quan trọng cho track:

if not cap.isOpened():
    raise RuntimeError(f"Không mở được webcam ở index {CAM_INDEX}. Thử đổi sang 1, 2...")
```

Giải thích:

- CAP_DSHOW (DirectShow) trên Windows ổn định hơn CAP_MSMF .
- CAP_PROP_BUFFERSIZE = 1 : chỉ giữ 1 frame trong bộ đệm → luôn lấy frame mới nhất, tránh độ trễ làm động cơ bám sai.

3.2. Đọc 1 khung hình trong vòng lặp

```
ret, frame = cap.read()
if not ret or frame is None:
    print("[WARN] Không đọc được frame, thử lại...")
    continue

h, w = frame.shape[:2]    # chiều cao, rộng ảnh
frame_cx = w // 2        # tâm ảnh theo X
frame_cy = h // 2        # tâm ảnh theo Y
```

Giải thích: `cap.read()` trả về `(ret, frame)`. `ret=False` nghĩa là đọc lỗi (rút cam, cam bận...) → bỏ qua frame đó. `frame` là ảnh dạng mảng numpy `(H, W, 3)` (BGR).

4. CODE PHÁT HIỆN ĐỐI TƯỢNG (YOLO)

File [detector.py](#) — dùng model YOLOv11-segmentation đã huấn luyện (`best_seg.pt`).

4.1. Vá lỗi PyTorch 2.6 (weights_only)

```
import torch
# PyTorch 2.6 đổi mặc định weights_only=True -> không load được model YOLO.
# Cho phép các class của ultralytics vào allowlist để torch.load chạy được.
try:
    from ultralytics.nn.tasks import (
        SegmentationModel, DetectionModel, PoseModel, ClassificationModel,
    )
    torch.serialization.add_safe_globals([
        SegmentationModel, DetectionModel, PoseModel, ClassificationModel,
    ])
except Exception as e:
    print("[WARN] không thêm được safe_globals:", e)
```

4.2. Class MouseDetector

```
class MouseDetector:
    def __init__(self, model_path="best_seg.pt", conf=DET_CONF, imgsz=320):
        self.model = YOLO(model_path)  # nạp model
        self.conf = conf                # ngưỡng tin cậy (0.5 = chắc chắn 50% mới tính)
        self.imgsz = imgsz              # kích thước input (320 nhanh, 640 chính xác hơn)

    def detect(self, frame):
        """Trả về list các đối tượng phát hiện được, mỗi cái là 1 dict."""
        results = self.model(frame, conf=self.conf, imgsz=self.imgsz, verbose=False)
        dets = []
        if not results:
            return dets

        r = results[0]
        if r.bboxes is None or len(r.bboxes) == 0:
            return dets

        h, w = frame.shape[:2]

        # Lấy mask nếu là model segmentation
        masks = None
        if r.masks is not None:
            masks = r.masks.data.cpu().numpy()  # (N, mh, mw)

        boxes = r.bboxes.xyxy.cpu().numpy()  # tọa độ hộp (N, 4)
        confs = r.bboxes.conf.cpu().numpy()  # độ tin cậy (N,)

        for i in range(len(boxes)):
            x1, y1, x2, y2 = (int(v) for v in boxes[i])
            conf = float(confs[i])

            # Tâm mặc định = tâm hộp
            cx, cy = (x1 + x2) // 2, (y1 + y2) // 2

            mask_bool = None
            if masks is not None and i < len(masks):
                m = masks[i]
                if m.shape[:2] != (h, w):
                    m = cv2.resize(m, (w, h), interpolation=cv2.INTER_NEAREST)
                mask_bool = m > 0.5
            # Tâm chính xác hơn = centroid của vùng mask (giữa thân chuột)
```

```

ys, xs = np.where(mask_bool)
if len(xs) > 0:
    cx = int(xs.mean())
    cy = int(ys.mean())

dets.append({
    "box": (x1, y1, x2, y2),
    "center": (cx, cy),
    "conf": conf,
    "mask": mask_bool,
})
return dets

```

Giải thích: Mỗi đối tượng trả về 1 dict gồm:

- box : 4 tọa độ góc hộp bao (x1, y1, x2, y2) .
- center : tâm — nếu có mask thì lấy **trọng tâm vùng mask** (chính xác hơn tâm hộp).
- conf : độ tin cậy (0..1).
- mask : mảng True/False đánh dấu các pixel thuộc con chuột.

4.3. Gọi phát hiện trong vòng lặp

```

detector = MouseDetector(conf=0.5)
...
detections = detector.detect(frame)

```

5. CODE ĐIỀU KHIỂN PAN/TILT (ĐỘNG CƠ BƯỚC)

Mỗi trục (ngang = pan, dọc = tilt) là 1 đối tượng `StepperAxis` . Ý tưởng: **tốc độ quay tỉ lệ với sai số** — chuột càng xa tâm thì quay càng nhanh, càng gần tâm thì quay càng chậm để không vọt qua (overshoot).

5.1. Class StepperAxis

```
class StepperAxis:
    def __init__(self, axis_letter, deadzone, max_error, max_sps, min_sps):
        self.axis      = axis_letter    # 'P' = pan, 'T' = tilt
        self.deadzone   = deadzone      # sai số nhỏ hơn ngưỡng này -> coi như đã trúng -> dừng
        self.max_error  = max_error     # sai số >= ngưỡng này -> chạy hết tốc
        self.max_sps    = max_sps       # tốc độ tối đa (step/s)
        self.min_sps    = min_sps       # tốc độ tối thiểu (sát tâm bò chậm)
        self.last_sps_sent = None
        self.last_send_t  = 0.0
        self.current_sps  = 0           # tốc độ hiện tại (để debug + tính tọa độ)

    def _sps_from_error(self, error):
        """Tính tốc độ (có dấu) từ sai số pixel."""
        abs_err = abs(error)
        if abs_err <= self.deadzone:
            return 0                    # trong vùng chết -> dừng
        if abs_err >= self.max_error:
            mag = self.max_sps          # quá xa -> hết tốc
        else:
            # nội suy tuyến tính giữa min_sps và max_sps
            ratio = (abs_err - self.deadzone) / (self.max_error - self.deadzone)
            mag = self.min_sps + (self.max_sps - self.min_sps) * ratio
        sps = int(mag)
        return sps if error > 0 else -sps    # giữ dấu theo hướng sai số
```

5.2. Hàm cập nhật tốc độ khi BẮM mục tiêu

```
def update(self, error, send_raw, force_stop=False,
           block_pos=False, block_neg=False):
    now = time.time()
    sps = 0 if force_stop else self._sps_from_error(error)

    # CHẶN KÉP: nếu đã chạm công tắc theo hướng đang muốn đi -> ép dừng
    if sps > 0 and block_pos:
        sps = 0
    elif sps < 0 and block_neg:
        sps = 0

    self.current_sps = sps

    # Chỉ gửi serial khi tốc độ thay đổi đáng kể -> đỡ spam Arduino
    changed_enough = (
        self.last_sps_sent is None
        or (sps == 0 and self.last_sps_sent != 0)      # vừa mới dừng
        or (sps != 0 and self.last_sps_sent == 0)      # vừa mới chạy
        or abs(sps - self.last_sps_sent) >= SPS_SEND_STEP
    )
    if changed_enough and (now - self.last_send_t) >= SEND_THROTTLE:
        send_raw(f"{self.axis}{sps}\n")
        self.last_sps_sent = sps
        self.last_send_t = now
```

5.3. Hàm gửi tốc độ thẳng (dùng cho quét)

```
def send_sps(self, sps, send_raw, block_pos=False, block_neg=False):
    """Gửi tốc độ trực tiếp (dùng cho Scanner). Có chặn công tắc."""
    now = time.time()
    sps = int(sps)
    if sps > 0 and block_pos:
        sps = 0
    elif sps < 0 and block_neg:
        sps = 0
    self.current_sps = sps
    # ... (logic gửi giống update, chỉ gửi khi đổi đủ lớn)
    send_raw(f"{self.axis}{sps}\n")
```

5.4. Khởi tạo 2 trục + cách tính sai số

```
# Trục ngang (pan): dx > 0 -> chuột bên PHẢI -> camera quay phải (P dương)
axis_x = StepperAxis("P", deadzone=15, max_error=450, max_sps=1800, min_sps=160)
# Trục dọc (tilt): dy > 0 -> chuột phía DƯỚI -> camera cúi xuống (T dương)
axis_y = StepperAxis("T", deadzone=15, max_error=380, max_sps=1500, min_sps=160)

# Trong vòng lặp, sau khi có obj_cx, obj_cy (tâm chuột) và aim_x, aim_y (điểm ngắm):
dx = obj_cx - aim_x      # >0: chuột bên phải tâm ngắm
dy = obj_cy - aim_y      # >0: chuột phía dưới tâm ngắm

axis_x.update(dx, send_raw, block_pos=lim_pan_pos, block_neg=lim_pan_neg)
axis_y.update(dy, send_raw, block_pos=lim_tilt_pos, block_neg=lim_tilt_neg)
```

Giải thích: dx, dy là sai số pixel giữa tâm chuột và điểm ngắm. Hàm update biến sai số thành tốc độ step/s rồi gửi cho Arduino. Khi $|dx| \leq \text{deadzone}$ và $|dy| \leq \text{deadzone}$ thì cả 2 trục đều dừng → camera đã ngắm trúng.

6. CODE QUÉT TUẦN TRA (SCANNER)

Khi mất mục tiêu quá 2 giây, camera tự quét trái↔phải để tìm lại. Vùng quét giới hạn bằng **toạ độ** (đếm số step) chứ không dựa vào công tắc (vì công tắc chậm chạp).

6.1. Class Scanner

```
class Scanner:
    def __init__(self):
        self.active = False
        self.pan_dir = +1          # +1 = phải, -1 = trái
        self.tilt_dir = +1        # +1 = xuống, -1 = lên
        self.tilt_nudge_until = 0.0
        self.lost_since = None     # thời điểm bắt đầu mất mục tiêu
        self._pan_flip_lock_until = 0.0

    def on_target_found(self):
        """Có lại mục tiêu -> tắt quét."""
        self.active = False
        self.lost_since = None

    def on_target_lost(self):
        """Mất mục tiêu -> ghi nhận thời điểm để chờ rồi quét."""
        if self.lost_since is None:
            self.lost_since = time.time()

    def should_scan(self):
        """Đã mất mục tiêu đủ lâu (>= 2s) để bắt đầu quét chưa?"""
        if self.lost_since is None:
            return False
        return (time.time() - self.lost_since) >= SCAN_START_DELAY
```

6.2. Hàm tính tốc độ quét (theo toạ độ)

```
def compute(self, now, lim_pan_neg, lim_pan_pos, lim_tilt_neg, lim_tilt_pos,
            pan_pos=0.0):
    """Quét pan trong vùng [PAN_LIMIT_MIN, PAN_LIMIT_MAX]. Đảo chiều ở mép."""
    self.active = True

    flip = False
    if USE_COORD_SCAN:
        # Quét thuần theo toạ độ - bỏ qua công tắc (tránh kẹt cứng)
        if self.pan_dir > 0 and pan_pos >= PAN_LIMIT_MAX:
            flip = True
        elif self.pan_dir < 0 and pan_pos <= PAN_LIMIT_MIN:
            flip = True

    if flip and now >= self._pan_flip_lock_until:
        self.pan_dir = -self.pan_dir          # đảo chiều
        self.tilt_nudge_until = now + SCAN_TILT_STEP_TIME # nhích tilt 1 bước
        self._pan_flip_lock_until = now + 0.4    # khoá 0.4s tránh rung

    pan_sps = SCAN_PAN_SPS * self.pan_dir      # tốc độ pan khi quét

    # TILT: chỉ nhích 1 chút mỗi khi pan đổi chiều
    if now < self.tilt_nudge_until:
        if (self.tilt_dir > 0 and lim_tilt_pos) or (self.tilt_dir < 0 and lim_tilt_neg):
            self.tilt_dir = -self.tilt_dir
            tilt_sps = SCAN_TILT_SPS * self.tilt_dir
        else:
            tilt_sps = 0

    return pan_sps, tilt_sps
```

Giải thích: Camera quét hình chữ S — pan đi qua đi lại, mỗi lần chạm mép vùng (theo toạ độ `pan_pos`) thì đảo chiều và nhích tilt lên/xuống một chút để quét hết khung. Toạ độ `pan_pos` được tính bằng cách tích phân tốc độ (xem mục 8.3).

7. CODE BẮN LASER

Yêu cầu: chỉ bắn khi chuột đã vào vùng chết **VÀ động cơ đã dừng hẳn** (không còn xoay), giữ ổn định ≥ 0.4 giây rồi mới bắn 1 phát 3.5 giây. Phải ra khỏi vùng rồi vào lại mới bắn phát tiếp.

7.1. Hàm bật/tắt laser

```
laser_on = False
```

```
def set_laser(on):  
    """Chỉ gửi serial khi trạng thái laser thật sự đổi (tránh spam)."""  
    global laser_on  
    if on and not laser_on:  
        send("L")          # bật  
        laser_on = True  
    elif (not on) and laser_on:  
        send("K")          # tắt  
        laser_on = False
```

7.2. Logic bắn (state machine trong vòng lặp)

```
# ===== Cấu hình =====
LASER_BURST = 3.5      # bắn 3.5 giây mỗi lần
SETTLE_TIME = 0.4      # phải đứng yên 0.4s rồi mới bắn
laser_armed = True     # đã " nạp đạn " cho phát kế tiếp chưa
laser_fire_until = 0.0 # thời điểm tắt laser (đang trong burst)
dz_enter_t = None      # thời điểm chuột bắt đầu vào vùng

# ===== Trong vòng lặp =====
target_in_deadzone = target_found and center_in_deadzone
_now_laser = time.time()
motor_stopped = (axis_x.current_sps == 0 and axis_y.current_sps == 0)

if target_in_deadzone:
    if dz_enter_t is None:
        dz_enter_t = _now_laser
    # Đã đứng yên (motor dừng) đủ lâu chưa?
    settled = motor_stopped and (_now_laser - dz_enter_t >= SETTLE_TIME)

    if laser_armed and settled:
        laser_fire_until = _now_laser + LASER_BURST    # bắt đầu bắn 3.5s
        laser_armed = False
        set_laser(True)
    elif (not laser_armed) and _now_laser < laser_fire_until:
        set_laser(True)    # đang trong burst -> giữ sáng
    else:
        set_laser(False)    # chưa settle, hoặc hết burst -> tắt

    # Nếu động cơ chạy lại (chuột nhúc nhích) -> reset đồng hồ đợi
    if not motor_stopped:
        dz_enter_t = _now_laser
else:
    set_laser(False)    # ra khỏi vùng -> tắt
    laser_armed = True    # nạp lại cho phát sau
    dz_enter_t = None
```

Giải thích từng điều kiện:

1. target_in_deadzone : chuột nằm trong ô vuông deadzone quanh điểm ngắm.
2. motor_stopped : cả 2 trục đều có tốc độ = 0 → camera đã dừng hẳn.
3. settled : đã giữ trạng thái đứng yên ≥ 0.4 giây.

4. `laser_armed` : chờ chống bắn liên tục — chỉ bắn 1 phát, muốn bắn lại phải ra khỏi vùng rồi vào lại (lúc đó `laser_armed` được nạp `True` trở lại).

8. VÒNG LẶP CHÍNH

Ghép tất cả lại. Mỗi vòng lặp xử lý 1 khung hình.

8.1. Khung sườn vòng lặp

```
while True:
    # 1) Đọc frame
    ret, frame = cap.read()
    if not ret or frame is None:
        continue
    h, w = frame.shape[:2]
    frame_cx, frame_cy = w // 2, h // 2
    aim_x = frame_cx + AIM_OFFSET_X    # điểm ngắm laser (= tâm + offset hiệu chỉnh)
    aim_y = frame_cy + AIM_OFFSET_Y

    # 2) Phát hiện
    detections = detector.detect(frame)

    # 3) Nhớ tạm detection cũ vài frame (chống nhấp nháy khi YOLO miss 1-2 frame)
    if detections:
        last_dets, miss_count, display_dets, fresh = detections, 0, detections, True
    else:
        miss_count += 1
        display_dets = last_dets if miss_count <= DET_PERSIST_FRAMES else []
        fresh = False
```

8.2. Chọn mục tiêu + tính sai số (có làm mượt EMA)

```
target_found = False
if display_dets:
    # Chọn con GẦN TÂM nhất
    det = min(display_dets,
               key=lambda d: (d["center"][0]-frame_cx)**2 + (d["center"][1]-frame_cy)**2)
    x1, y1, x2, y2 = det["box"]

    # Điểm ngắm: CẠNH TRÊN giữa box (để laser trúng đầu chuột)
    if AIM_AT_TOP:
        raw_cx, raw_cy = (x1 + x2) // 2, y1 + AIM_TOP_MARGIN
    else:
        raw_cx, raw_cy = det["center"]

    # Làm mượt EMA -> chống box nhảy tung tưng -> camera đỡ giật
    if smooth_cx is None:
        smooth_cx, smooth_cy = float(raw_cx), float(raw_cy)
    else:
        smooth_cx = SMOOTH_ALPHA * raw_cx + (1 - SMOOTH_ALPHA) * smooth_cx
        smooth_cy = SMOOTH_ALPHA * raw_cy + (1 - SMOOTH_ALPHA) * smooth_cy
    obj_cx, obj_cy = int(smooth_cx), int(smooth_cy)

    target_found = True
    dx = obj_cx - aim_x
    dy = obj_cy - aim_y
    if INVERT_PAN: dx = -dx
    if INVERT_TILT: dy = -dy
    center_in_deadzone = (abs(dx) <= DEADZONE_X and abs(dy) <= DEADZONE_Y)
```

EMA (Exponential Moving Average) = lọc làm mượt: tâm mới = 50% giá trị đo + 50% giá trị cũ. Giúp camera không giật theo box YOLO nhảy mỗi frame.

8.3. Điều khiển động cơ + đếm tọa độ pan

```
poll_limits()    # đọc trạng thái công tắc

if target_found:
    # BẮM theo
    if _current_mode != "TRACK":
        send_raw("F"); _current_mode = "TRACK"
    scanner.on_target_found()
    axis_x.update(dx, send_raw, block_pos=lim_pan_pos, block_neg=lim_pan_neg)
    axis_y.update(dy, send_raw, block_pos=lim_tilt_pos, block_neg=lim_tilt_neg)
else:
    # QUÉT tìm lại
    scanner.on_target_lost()
    if scanner.should_scan():
        if _current_mode != "SCAN":
            send_raw("S"); _current_mode = "SCAN"
        pan_scan, tilt_scan = scanner.compute(
            time.time(), lim_pan_neg, lim_pan_pos, lim_tilt_neg, lim_tilt_pos,
            pan_pos=pan_pos_steps)
        axis_x.send_sps(pan_scan, send_raw)
        axis_y.send_sps(tilt_scan, send_raw)
    else:
        axis_x.send_sps(0, send_raw)    # chờ chút trước khi quét
        axis_y.send_sps(0, send_raw)

# Đếm tọa độ pan = tích phân tốc độ theo thời gian
now = time.time()
dt = now - prev_t
if dt > 0:
    pan_pos_steps += axis_x.current_sps * dt    # vị trí (step) = ∫ tốc độ dt
prev_t = now
```

8.4. Phím tắt hiệu chỉnh (chạy trong vòng lặp)

```
key = cv2.waitKey(1) & 0xFF
if key == 27:                # ESC -> thoát
    break
elif key == ord('i'): AIM_OFFSET_Y -= AIM_CAL_STEP    # dịch tâm ngắm LÊN
elif key == ord('k'): AIM_OFFSET_Y += AIM_CAL_STEP    # XUỐNG
elif key == ord('j'): AIM_OFFSET_X -= AIM_CAL_STEP    # TRÁI
elif key == ord('l'): AIM_OFFSET_X += AIM_CAL_STEP    # PHẢI
elif key == ord('r'): AIM_OFFSET_X = AIM_OFFSET_Y = 0 # reset
elif key == ord('o'):                # LƯU offset ra file
    open(AIM_OFFSET_FILE, "w").write(f"{AIM_OFFSET_X},{AIM_OFFSET_Y}")
elif key == ord('h'): pan_pos_steps = 0.0            # đặt vị trí pan hiện tại = 0
elif key == ord('['): PAN_LIMIT_MIN = int(pan_pos_steps) # mép trái vùng tuần tra
elif key == ord(']'): PAN_LIMIT_MAX = int(pan_pos_steps) # mép phải vùng tuần tra
```

8.5. Dọn dẹp khi thoát

```
finally:
    set_laser(False)    # tắt laser
    send("x")           # dừng hết động cơ
    time.sleep(0.2)
    ser.close()         # đóng cổng Serial
    cap.release()       # giải phóng camera
    cv2.destroyAllWindows()
```

9. CODE PHÍA ARDUINO (.ino)

Arduino nhận lệnh tốc độ qua Serial và phát xung STEP/DIR cho 2 driver HBS57.

9.1. Khai báo chân

```
#define PAN_STEP    2    // xung bước trục PAN
#define PAN_DIR     3    // hướng trục PAN
#define TILT_STEP   5
#define TILT_DIR    6

#define LASER       A5   // laser (active HIGH)

// Công tắc hành trình (chân chung GND, INPUT_PULLUP, nhấn = LOW)
#define LIM_TILT_NEG A0   // ngắt khi quay LÊN
#define LIM_TILT_POS A1   // ngắt khi quay XUỐNG
#define LIM_PAN_NEG  A3   // ngắt khi quay TRÁI
#define LIM_PAN_POS  A2   // ngắt khi quay PHẢI

const long MAX_SPS = 4000; // tốc độ tối đa
const long MIN_SPS = 150;  // dưới ngưỡng này coi như dừng
```

9.2. Áp dụng lệnh tốc độ

```
void applyCmd(char axis, long val) {
    if (axis == 'P') {
        pan_sps = constrain(val, -MAX_SPS, MAX_SPS);
        // pan_sps > 0 = quay PHẢI (đã đảo HIGH/LOW cho khớp phần cứng)
        if (pan_sps > 0) digitalWrite(PAN_DIR, LOW);
        else if (pan_sps < 0) digitalWrite(PAN_DIR, HIGH);
    } else if (axis == 'T') {
        tilt_sps = constrain(val, -MAX_SPS, MAX_SPS);
        if (tilt_sps > 0) digitalWrite(TILT_DIR, HIGH);
        else if (tilt_sps < 0) digitalWrite(TILT_DIR, LOW);
    }
}
```

9.3. Đọc công tắc (chống nhiễu)

```
bool limitHit(int pin) {  
    if (digitalRead(pin) == LOW) {           // nhấn = LOW  
        delayMicroseconds(50);  
        if (digitalRead(pin) == LOW) {       // đọc lại 2 lần để chắc  
            delayMicroseconds(50);  
            if (digitalRead(pin) == LOW) return true;  
        }  
    }  
    return false;  
}
```

9.4. Phát xung bước (trong loop)

```
void loop() {  
    // ... đọc lệnh Serial (P/T/L/K/x/F/S/M) ...  
  
    unsigned long now = micros();  
  
    // PAN: chạm công tắc hướng đang đi -> KHÔNG phát xung hướng đó  
    long pan_abs = labs(pan_sps);  
    bool pan_blocked = (pan_sps > 0 && limitHit(LIM_PAN_POS)) ||  
                      (pan_sps < 0 && limitHit(LIM_PAN_NEG));  
    if (pan_abs >= MIN_SPS && !pan_blocked) {  
        unsigned long half_us = 1000000UL / (2UL * pan_abs); // nửa chu kỳ xung  
        if (now - pan_last_us >= half_us) {  
            pan_pin_state = !pan_pin_state; // lật mức -> tạo xung vuông  
            digitalWrite(PAN_STEP, pan_pin_state);  
            pan_last_us = now;  
        }  
    }  
  
    // TILT: tương tự (chỉ chặn khi ở chế độ TRACK)  
    // ...  
}
```

Giải thích: Tốc độ step/s được biến thành tần số lật chân STEP. Mỗi half_us micro-giây thì lật chân 1 lần → tạo xung vuông. sps càng lớn → half_us càng nhỏ → xung càng nhanh → động cơ quay nhanh.

PHỤ LỤC: TỔNG HỢP DANH SÁCH HÀM

Hàm / Class	File	Chức năng
MouseDetector.__init__	detector.py	Nạp model YOLO
MouseDetector.detect	detector.py	Phát hiện chuột, trả box/center/conf/mask
send_raw	main.py	Gửi lệnh tốc độ tới Arduino
send	main.py	Gửi lệnh + log
poll_limits	main.py	Đọc trạng thái công tắc từ Arduino
set_laser	main.py	Bật/tắt laser
StepperAxis._sps_from_error	main.py	Tính tốc độ từ sai số pixel
StepperAxis.update	main.py	Cập nhật tốc độ khi bám mục tiêu
StepperAxis.send_sps	main.py	Gửi tốc độ thẳng (cho quét)
Scanner.on_target_found/lost	main.py	Bật/tắt trạng thái quét
Scanner.should_scan	main.py	Quyết định có nên quét chưa
Scanner.compute	main.py	Tính tốc độ pan/tilt khi quét
applyCmd	.ino	Áp dụng lệnh tốc độ Arduino
limitHit	.ino	Đọc công tắc (chống nhiễu)
loop	.ino	Vòng lặp phát xung STEP/DIR

Mã nguồn đầy đủ: xem các file [main.py](#), [detector.py](#),
và [arduino/motor_test/arduino_stepper_laser \(1\).ino](#).